

Variable-Structure Automaton Inference from Interaction Sequences

Engin Durmaz

May 16, 2026

Abstract

This report documents inference of a Variable-Structure Automaton (VSA) from a unique dataset of state–action–feedback tuples generated in a P-model environment. We give the count-based background, methodology for all six course tasks—dataset fingerprint, estimation, optimal-action identification, ε -optimality, simulation, perturbation, and a model-assumption check on per-state penalties—and describe how `main.py` runs them end-to-end. Implementation lives under `./proj_python/`; code excerpts are in Appendix A.

1 Introduction

The term project is **data-driven and reproducible**: from observed interaction sequences alone we must reconstruct the state–action policy, transition dynamics, penalty structure, and norm of behavior of an unknown Variable-Structure Automaton (VSA). The environment is a **P-model**: binary feedback $\beta \in \{0,1\}$ encodes favorable vs. unfavorable response, and the assignment assumes that the *global* penalty probability $c_k = P(\beta=1 \mid \alpha_k)$ depends on the action α_k but not on the source state ϕ_i (Task 6 tests this assumption empirically).

The system has five states $\{\phi_1, \dots, \phi_5\}$, three actions $\{\alpha_1, \alpha_2, \alpha_3\}$, and many sequences indexed by `sequence_id`. Each row of the CSV is one step:

$$(\text{sequence_id}, \text{step}, \Phi_{\text{src}}(n), \Phi_{\text{dst}}(n), \alpha(n), \beta(n)).$$

Before modeling we report a **dataset fingerprint** (row count, number of sequences, mean length, empirical penalty rate per action, checksum). The six tasks then proceed as follows:

Task 1: Estimation — empirical counts $N(\phi_i, \alpha_k)$, $N(\phi_i, \alpha_k, \phi_j)$, $N(\alpha_k, \beta)$; policy $P(\alpha_k \mid \phi_i)$, transitions $P(\phi_j \mid \phi_i, \alpha_k)$, and penalties c_k .

Task 2: Optimal action — $\alpha^* \in \arg \min_k c_k$.

Task 3: ε -optimality — $\varepsilon = 1 - \min_i P(\alpha^* \mid \phi_i)$ over visited states.

Task 4: Simulation — synthetic trajectories from the learned model; compare marginals and penalty rates to the real CSV.

Task 5: Perturbation — remove a random fraction of rows, refit Tasks 1–4, and assess stability of c_k , policy, and ε .

Task 6: Model assumption check — estimate $c_k(\phi_i) = P(\beta=1 \mid \alpha_k, \phi_i)$ per state and compare across ϕ_i (and, under perturbation, across trials) to judge the P-model.

All tasks are implemented in `../proj_python/`. The orchestrator `main.run_all` prints the fingerprint, runs Tasks 1–4 on the full dataset (baseline), optionally repeats Tasks 1–4 on row-subsampled copies when `perturbation_rate > 0`, summarizes Task 5 metrics, and prints combined Task 6 tables once at the end (baseline plus every perturbation trial when applicable). Each task module can also be run standalone and writes outputs under `outputs/`. This report aligns notation with the course assignment and with that code; Section 2 gives VSA context and empirical estimators (Section 2.3); Section 3 maps each task to code.

2 Background

2.1 Variable-structure automata and the P-model

A **Variable-Structure Automaton** (VSA) is a stochastic decision model in which an agent visits discrete states ϕ_i , chooses actions α_k , and receives environment feedback. In this project the state set is $\{\phi_1, \dots, \phi_5\}$, the action set is $\{\alpha_1, \alpha_2, \alpha_3\}$, and each interaction step is recorded in the dataset together with a sequence identifier and step index. The learning goal is to recover, from many such sequences only, four objects required by the assignment:

- a **state–action policy** $P(\alpha_k \mid \phi_i)$,
- **transition dynamics** $P(\phi_j \mid \phi_i, \alpha_k)$,
- an **environmental penalty structure** (probabilities of unfavorable feedback), and
- a **norm of behavior** (e.g. how often the optimal action is taken in each state, via ε -optimality).

The environment is a **P-model**: feedback $\beta \in \{0, 1\}$ indicates a favorable (0) or unfavorable (1) response. The working assumption is that the *global* penalty $c_k = P(\beta=1 \mid \alpha_k)$ depends on the action but not explicitly on the source state—so one penalty vector suffices for Tasks 1–5. Task 6 examines whether that assumption is consistent with the data when penalties are estimated separately in each state.

2.2 Approach used in this project

The implementation is **purely data-driven**: every quantity is an empirical frequency or a ratio of counts from the provided CSV, followed by row-wise normalization where conditional distributions are required. We do not fit parametric models, run gradient-based learning, or impose structure beyond what the counts support. Consistency checks (policy rows and transition rows sum to one; global c_k matches the sample mean of β given α) validate the arithmetic.

The six assignment tasks form a pipeline: a **dataset fingerprint** (Section 2.3) validates scale and raw penalty rates; **estimation** builds counts and conditional probabilities; **optimal-action** and **ε -optimality** analyses use the policy and global penalties; **simulation** draws synthetic trajectories from the learned model and compares marginals to the real data; **perturbation** randomly deletes rows and refits to test robustness; **model critique** compares per-state penalties $c_k(\phi_i)$ across ϕ_i to assess the P-model. Section 3 maps each step to code; Appendix A lists the corresponding excerpts.

2.3 Empirical estimators

All formulas below are evaluated on a (possibly subsampled) interaction table. Each CSV row is one step; we write $n \in R$ for row indices and S for the set of distinct `sequence_id` values. For a fixed sequence $s \in S$, J_s is how many rows belong to s (that sequence’s length in steps).

Dataset fingerprint. Before modeling, the assignment requires five summaries of the raw table:

- **Total number of rows** $|R|$ (one per step in the file).
- **Number of sequences** $|S|$ (distinct `sequence_id`).
- **Mean sequence length** $\bar{L} = \frac{1}{|S|} \sum_{s \in S} J_s$ (average of the per-sequence lengths J_s).
- **Empirical penalty rate per action** α_k : fraction of rows with $\alpha(n) = \alpha_k$ that have $\beta(n) = 1$,

$$\frac{1}{\#\{n \in R : \alpha(n) = \alpha_k\}} \sum_{\substack{n \in R \\ \alpha(n) = \alpha_k}} \beta(n).$$

- **Checksum** $C = \sum_{n \in R} \text{step}(n)$.

On the full dataset, each per-action rate above equals the global penalty c_k used in estimation (both are the sample mean of β given α_k).

Raw counts. The assignment requires these three tables *before* any probabilities are reported: they are the empirical basis for the policy $P(\alpha_k \mid \phi_i)$, the transitions $P(\phi_j \mid \phi_i, \alpha_k)$, and the penalties c_k . Each entry counts how many rows $n \in R$ match a pattern on `phi_src`, `phi_dst`, `alpha`, and `beta` (written $\Phi_{\text{src}}(n)$, $\Phi_{\text{dst}}(n)$, $\alpha(n)$, $\beta(n)$ below).

$N(\phi_i, \alpha_k)$ — **actions taken in a state.**

$$N(\phi_i, \alpha_k) = \#\{n \in R : \Phi_{\text{src}}(n) = \phi_i, \alpha(n) = \alpha_k\}. \quad (1)$$

$N(\phi_i, \alpha_k, \phi_j)$ — **transitions under an action.**

$$N(\phi_i, \alpha_k, \phi_j) = \#\{n \in R : \Phi_{\text{src}}(n) = \phi_i, \alpha(n) = \alpha_k, \Phi_{\text{dst}}(n) = \phi_j\}. \quad (2)$$

$N(\alpha_k, \beta)$ — **environment feedback per action.**

$$N(\alpha_k, \beta) = \#\{n \in R : \alpha(n) = \alpha_k, \beta(n) = \beta\}. \quad (3)$$

Write $r_i = \sum_k N(\phi_i, \alpha_k)$ for the row total over actions in state ϕ_i .

Conditional estimates (Tasks 1–4). For visited states ($r_i > 0$),

$$P(\alpha_k \mid \phi_i) = \frac{N(\phi_i, \alpha_k)}{r_i}. \quad (4)$$

For each action α_k , transition probabilities normalize each row of $N(\phi_i, \alpha_k, \phi_j)$ over ϕ_j (empty rows yield zeros). The **global** P-model penalty is

$$c_k = P(\beta=1 \mid \alpha_k) = \frac{N(\alpha_k, 1)}{N(\alpha_k, 0) + N(\alpha_k, 1)}. \quad (5)$$

Task 2 sets $\alpha^* \in \arg \min_k c_k$. Task 3 uses visited states $\mathcal{V} = \{\phi_i : r_i > 0\}$ and $\varepsilon = 1 - \min_{\phi_i \in \mathcal{V}} P(\alpha^* \mid \phi_i)$. Task 4 samples $\alpha \sim P(\cdot \mid \phi)$, $\phi' \sim P(\cdot \mid \phi, \alpha)$, and $\beta \sim \text{Bernoulli}(c_\alpha)$ along each real sequence length.

Per-state penalties (Task 6). To test action-only penalties, for each pair (ϕ_i, α_k) with $N(\phi_i, \alpha_k) > 0$,

$$c_k(\phi_i) = P(\beta=1 \mid \alpha_k, \phi_i) = \frac{N(\phi_i, \alpha_k, \beta=1)}{N(\phi_i, \alpha_k)}, \quad (6)$$

where $N(\phi_i, \alpha_k, \beta=1)$ counts penalized steps with that state and action. Under a strict P-model, $c_k(\phi_i)$ should vary little in ϕ_i for fixed k ; a diagnostic is $\max_i c_k(\phi_i) - \min_i c_k(\phi_i)$ per action.

3 Methodology

All runnable code lives in `../proj_python/`. Two execution modes exist: **orchestrated** (`python main.py` $\rightarrow$ `load_all` $\rightarrow$ `run_all`) and **standalone** (each `run_*` module with `output_dir` under `outputs/`). Formulas are in Section 2.3; Appendix A lists the matching source.

3.1 General flow

The end-to-end pipeline is implemented in `main.run_all` (Appendix A.6). It always validates the raw CSV, fits the VSA quantities on the full dataset, and optionally repeats the fit on random row subsamples to test stability. Per-state penalty tables are printed once at the end so the console log stays readable. Figure 1 summarizes `run_all` with decisions and loops; Algorithm 1 states the same logic in pseudocode. A standalone run executes only `RunTasks1To4` for a single module at a time, reading and writing CSVs under `outputs/`.

Algorithm 1 General flow (`main.run_all`)

Require: CSV path, `perturbation_rate`, `experiment_count`, `random_seed`

```

1:  $df_{full} \leftarrow \text{load\_vsa\_dataset}(\text{path})$  ▷ util.py
2: run_fingerprint(path) ▷  $|R|, |S|, \bar{L}$ , penalty rates,  $C$ 
3:  $baseline \leftarrow \text{RUNTASKS1TO4}(df_{full}, baseline)$ 
4:  $results \leftarrow [baseline]$ 
5: if perturbation_rate > 0 then
6:   for  $t = 1$  to experiment_count do
7:      $df_p \leftarrow \text{\_drop\_rows\_randomly}(df_{full}, \text{rate}, rng)$ 
8:     append RUNTASKS1TO4( $df_p, t$ ) to  $results$ 
9:   end for
10:  \_print\_experiments\_summary( $baseline$ , perturbed runs)
11: end if
12: \_print\_c\_k\_per\_state\_runs( $results$ )

13: function RUNTASKS1TO4( $df, label$ )
14:    $est \leftarrow \text{run\_estimation}(df, \text{print\_penalty\_per\_state}=\text{False})$ 
15:    $opt \leftarrow \text{run\_optimal\_action}(est)$ 
16:    $eps \leftarrow \text{run\_epsilon\_optimality}(est, opt)$ 
17:    $sim \leftarrow \text{run\_simulation}(est, path)$ 
18:   return PipelineRunResult( $label, est, opt, eps, sim$ )
19: end function

```

3.2 Implementation details

The subsubsections below describe how each module realizes one part of the methodology. For each file we state the goal, point to the matching appendix listing, and list the main third-party libraries used (the standard library supplies `pathlib`, `dataclasses`, `argparse`, and `typing` where needed).

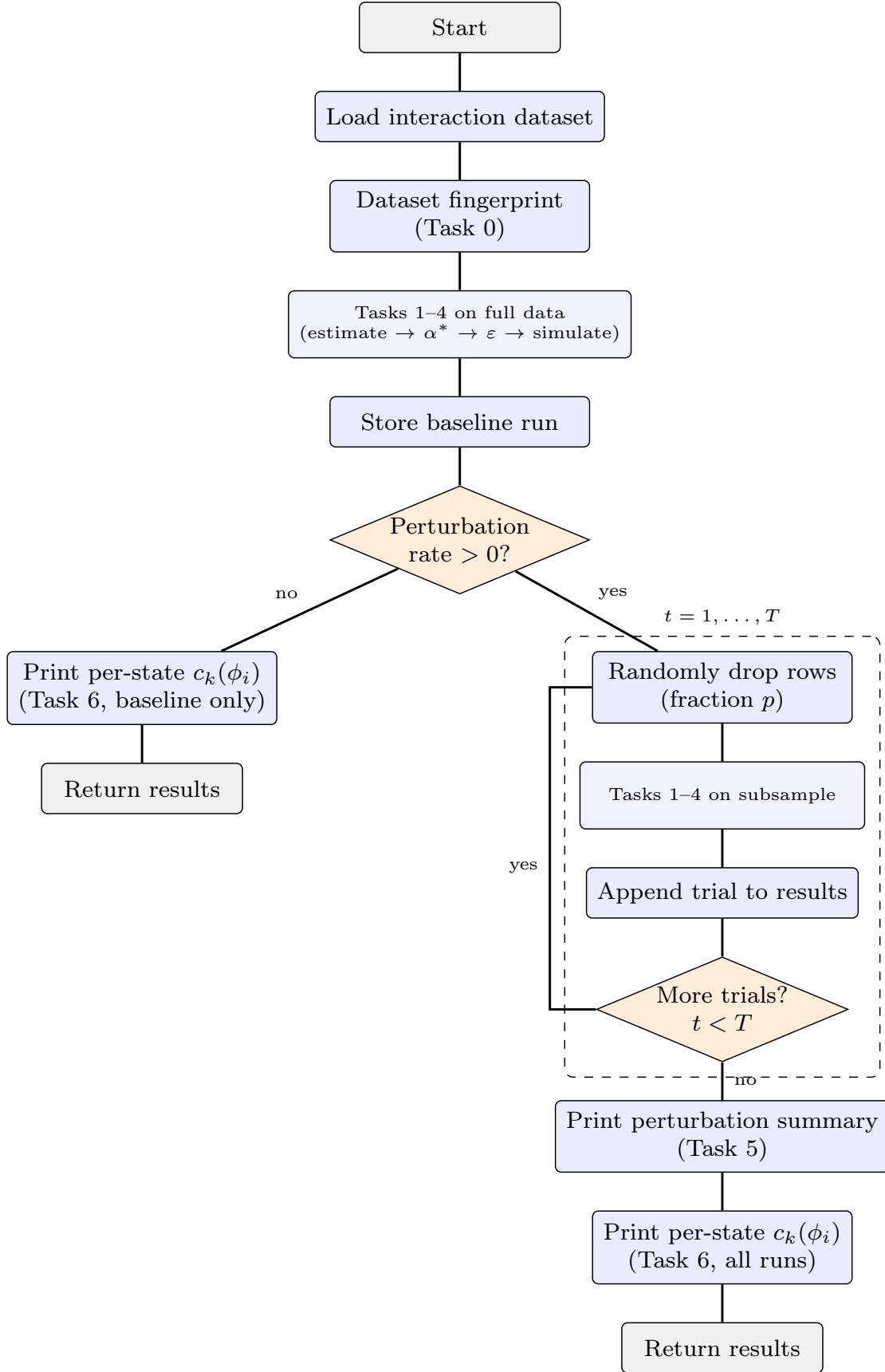


Figure 1: Flowchart for `run_all`: baseline Tasks 1–4 on the full dataset, optional perturbation loop over T trials (random row removal, refit, append), then Task 5 aggregate reporting when perturbation is enabled and Task 6 per-state penalty tables (baseline only if $p = 0$, else all stored runs).

3.2.1 Software map

Table 1 lists each project module, its main entry point(s), return type, and the matching appendix listing (file by file, no task labels).

Table 1: Source modules in `../proj_python/`. An orchestrated run (`python main.py`) uses every row via `main.run_all`; standalone runs call one `run_*` function at a time and pass CSVs through `outputs/`.

Module	Entry point(s)	Return type	Ref.
<code>fingerprint.py</code>	<code>run_fingerprint</code>	Fingerprint-Result	A.1
<code>estimation.py</code>	<code>run_estimation</code>	Estimation-Result	A.2
<code>optimal_action_identification.py</code>	<code>run_optimal_action</code>	OptimalAction-Result	A.3
<code>e_optimality_analysis.py</code>	<code>run_epsilon_optimality</code>	Epsilon-Optimality-Result	A.4
<code>simulation.py</code>	<code>run_simulation</code>	Simulation-Result	A.5
<code>main.py</code>	<code>run_all</code> , <code>load_all</code>	PipelineRun-Result (list)	A.6
<code>util.py</code>	<code>load_vsa_dataset</code>	DataFrame	A.7

3.2.2 Dataset fingerprint (`fingerprint.py`)

Goal. Before modeling, report $|R|$, $|S|$, $\bar{L}$, empirical penalty rate per action, and checksum C (Section 2.3) so the submission can be validated against the assignment fingerprint.

Code. Appendix A.1 (`run_fingerprint`, lines 86–99), invoked at the start of Algorithm 1.

Libraries. `pandas` (`groupby`, `mean`, `sum`).

3.2.3 Estimation (`estimation.py`)

Goal. From the interaction table, build raw count matrices and derive $P(\alpha_k \mid \phi_i)$, $P(\phi_j \mid \phi_i, \alpha_k)$, global c_k , and per-state $c_k(\phi_i)$; run consistency checks (Section 2.3).

Code. Appendix A.2 (`run_estimation`, lines 104–189). From `main`, per-state penalties are computed but not printed (`print_penalty_per_state=False`). Standalone `estimation.py` writes CSVs under `outputs/task1_estimation/`.

Libraries. `pandas` (`groupby`, `unstack`, `normalization`); `NumPy` (`allclose` for checks).

3.2.4 Optimal action (`optimal_action_identification.py`)

Goal. Identify $\alpha^* \in \arg \min_k c_k$ from the penalty vector and record ties deterministically.

Code. Appendix A.3 (`run_optimal_action`, lines 109–128); uses `estimation.penalty` in memory or `penalty.csv` when run standalone.

Libraries. `pandas` (`sorting`, `framing` c_k); `NumPy` (`isclose` for tie detection).

3.2.5 ε -optimality (`e_optimality_analysis.py`)

Goal. Compute $\varepsilon = 1 - \min_{\phi_i \in \mathcal{V}} P(\alpha^* \mid \phi_i)$ over visited states only.

Code. Appendix A.4 (`_compute_core` / `run_epsilon_optimality`, lines 110–130).

Libraries. `pandas` (policy and count matrices); `NumPy` (`min`, `isclose`).

3.2.6 Simulation (simulation.py)

Goal. Generate synthetic trajectories with the learned policy, transitions, and Bernoulli penalties; compare data vs. simulated marginals and penalty rates.

Code. Appendix A.5 (`_simulate_block`, `run_simulation`, lines 134–184).

Libraries. `pandas` (marginals, alignment); `NumPy` (`default_rng`, sampling, binomial draws).

3.2.7 Perturbation experiment (main.py)

Goal. Delete a random fraction of rows, refit the model on each subsample, and compare α^* , ε , and simulation L1 to the baseline (Algorithm 1, perturbation branch).

Code. Appendix A.6: `_drop_rows_randomly` (lines 36–51), `run_all` perturbation loop (lines 319–344), `_print_experiments_summary` (lines 162–255).

Libraries. `NumPy` (`default_rng`, choice for row indices); `pandas` (`drop`, `reset_index`).

3.2.8 Model assumption check (estimation.py, main.py)

Goal. Form $c_k(\phi_i)$ for each state–action pair and compare across states (spread diagnostic); optionally across perturbation trials (Section 2.3).

Code. `penalty_per_state` in Appendix A.2; printing and spread helpers in Appendix A.6 (`_spread_across_states`, `_print_c_k_per_state_runs`, lines 70–78, 150–159).

Libraries. `pandas` (per-state table); `NumPy` (max/min spread, absolute differences). Also see `_run_tasks_1_to_4` and `run_all` in Appendix A.6 (lines 91–133, 302–346).

3.2.9 Shared data loading (util.py)

Goal. Load the headerless interaction CSV into a typed `pandas.DataFrame` and define shared result containers passed between modules.

Code. Appendix A.7: column constants, `EstimationResult`, `OptimalActionResult`, and `load_vsa_dataset`.

Libraries. `pandas` (`read_csv`, frame types).

4 Experiments

Data. The provided file `../Engin-vsa_dataset_full.csv` (one row per step; Greek state/action labels) is the default input for `estimation.py`.

Reproducibility. From `../proj_python/`:

1. Task 1 only (writes CSVs and `report.txt`): `pythonestimation.py`
2. Task 2 after Task 1 files exist: `pythonoptimal_action_identification.py`
3. Task 3 after Tasks 1–2 outputs exist: `pythone_optimality_analysis.py`
4. Task 4 after Task 1 (and real CSV for marginals): `pythonsimulation.py`
5. End-to-end (`main.py`, `load_all`): `pythonmain.py`.
Defaults: full CSV; `perturbation_rate=20`, `random_seed=42`.

Console logs from `python main.py` include the fingerprint, full Task 1–4 reports per run, Task 5 aggregates, and combined Task 6 per-state penalty tables.

5 Conclusion

We summarized VSA background, a count-based approach, and estimators in Section 2. The pipeline recovers policy, transitions, and global penalties (Task 1), identifies α^* (Task 2), quantifies ε -optimality over visited states (Task 3), and validates a simulator against data marginals (Task 4). Task 5 stress-tests robustness under random row removal; Task 6 compares $c_k(\phi_i)$ across states to assess the P-model. Together, these six tasks satisfy the assignment structure from fingerprint through model critique.

A Supplementary source code listing

Excerpts below are taken from `../proj_python/`, one subsection per file. Line ranges match the repository at report time; together they implement the flow in Section 3.1 (Algorithm 1) and the estimators in Section 2.3.

A.1 fingerprint.py

```
df = load_vsa_dataset(csv_path)

n_rows = len(df)
n_sequences = int(df[CLMN_SEQ_ID].nunique())
lengths = df.groupby(CLMN_SEQ_ID, sort=False).size()
mean_sequence_length = float(lengths.mean())

penalty_rate_per_action = (
    df.groupby(CLMN_ALPHA)[CLMN_BETA].mean().sort_index()
)
penalty_rate_per_action.index.name = CLMN_ALPHA
penalty_rate_per_action.name = "empirical_penalty_rate"

checksum = int(df[CLMN_STEP].sum())
```

A.2 estimation.py

```
# count by grouping how many action alpha taken in state phi
N_phi_alpha = (
    df.groupby([CLMN_PHI_SRC, CLMN_ALPHA]).size()
    .unstack(fill_value=0)
    .reindex(index=states, columns=actions, fill_value=0)
)
N_phi_alpha.index.name = CLMN_PHI_SRC
N_phi_alpha.columns.name = CLMN_ALPHA

# count by grouping how many transitions from state phi_src to state phi_dst for action a
N_phi_alpha_phi: Dict[str, pd.DataFrame] = {}
for a in actions:
    tab = (
        df[df[CLMN_ALPHA] == a]
        .groupby([CLMN_PHI_SRC, CLMN_PHI_DST]).size()
        .unstack(fill_value=0)
        .reindex(index=states, columns=states, fill_value=0)
    )
    tab.index.name = CLMN_PHI_SRC
    tab.columns.name = CLMN_PHI_DST
    N_phi_alpha_phi[a] = tab

# count by grouping how many action alpha valuated with beta
```



```

N_alpha_beta = (
    df.groupby([CLMN_ALPHA, CLMN_BETA]).size()
    .unstack(fill_value=0)
    .reindex(index=actions, columns=betas, fill_value=0)
)
N_alpha_beta.index.name = CLMN_ALPHA
N_alpha_beta.columns.name = CLMN_BETA

# Conditional probabilities -----
# N_phi_alpha : how many action alpha taken in state phi
# N_phi_alpha_phi : how many transitions from state phi_src to state phi_dst for action a
# N_alpha_beta : how many action alpha valuated with beta

row_totals_phi = N_phi_alpha.sum(axis=1)
policy = N_phi_alpha.div(row_totals_phi.replace(0, np.nan), axis=0).fillna(0.0)

transition: Dict[str, pd.DataFrame] = {}
for a in actions:
    N = N_phi_alpha_phi[a]
    T = N.div(N.sum(axis=1).replace(0, np.nan), axis=0).fillna(0.0)
    transition[a] = T

total_per_action = N_alpha_beta.sum(axis=1).replace(0, np.nan)
penalty = (N_alpha_beta[1] / total_per_action).fillna(0.0)
penalty.name = "c_k"

N_phi_alpha_beta = (
    df.groupby([CLMN_PHI_SRC, CLMN_ALPHA, CLMN_BETA]).size()
    .unstack(fill_value=0)
    .reindex(columns=betas, fill_value=0)
)
totals_phi_alpha = N_phi_alpha_beta.sum(axis=1).replace(0, np.nan)
penalty_per_state = (
    (N_phi_alpha_beta[1] / totals_phi_alpha)
    .fillna(0.0)
    .unstack(CLMN_ALPHA)
    .reindex(index=states, columns=actions, fill_value=0.0)
)
penalty_per_state.index.name = CLMN_PHI_SRC
penalty_per_state.columns.name = CLMN_ALPHA

empirical_penalty = (
    df.groupby(CLMN_ALPHA)[CLMN_BETA].mean().reindex(actions, fill_value=0.0)
)
empirical_penalty.name = "empirical_penalty"

# Consistency checks -----
nonzero_phi = row_totals_phi > 0
policy_rows_ok = bool(
    np.allclose(policy.loc[nonzero_phi].sum(axis=1).values, 1.0, atol=1e-9)
)

transition_rows_ok = True
for a, T in transition.items():
    N = N_phi_alpha_phi[a]
    nonzero = N.sum(axis=1) > 0
    if not np.allclose(T.loc[nonzero].sum(axis=1).values, 1.0, atol=1e-9):
        transition_rows_ok = False
        break

penalty_matches_empirical = bool(
    np.allclose(penalty.values, empirical_penalty.values, atol=1e-9)
)

```

A.3 optimal_action_identification.py

```
c_min = float(penalty.min())
ties: List[str] = [str(a) for a in penalty.index[np.isclose(penalty.values, c_min)]]
alpha_star = ties[0] # deterministic pick: first action tied at the minimum

ranking = (
    penalty.sort_values(ascending=True)
        .to_frame()
        .assign(rank=lambda d: np.arange(1, len(d) + 1, dtype=int))
)
ranking["is_optimal"] = np.isclose(ranking["c_k"].values, c_min)
ranking.index.name = CLMN_ALPHA

result = OptimalActionResult(
    alpha_star=alpha_star,
    c_star=c_min,
    ties=ties,
    ranking=ranking,
    penalty=penalty,
    source=source,
)
```

A.4 e_optimality_analysis.py

```
def _compute_core(
    policy: pd.DataFrame,
    counts_phi_alpha: pd.DataFrame,
    alpha_star: str,
) -> Tuple[float, float, pd.Series, List[str]]:
    if alpha_star not in policy.columns:
        raise ValueError(
            f"alpha* column {alpha_star!r} not in policy columns {list(policy.columns)}. "
            "Regenerate policy.csv / run Task 1 after Task 2."
        )
    row_totals = counts_phi_alpha.sum(axis=1)
    visited = row_totals > 0
    if not bool(visited.any()):
        raise ValueError("No visited states in count matrix; cannot compute epsilon.")

    p_col = policy.loc[visited, alpha_star].astype(float)
    min_p = float(np.min(p_col.values))
    epsilon = 1.0 - min_p
    tol = 1e-9
    argmin_states = [str(s) for s in p_col.index[np.isclose(p_col.values, min_p, atol=tol)]]
    return epsilon, min_p, p_col, argmin_states
```

A.5 simulation.py

```
def _simulate_block(
    real_df: pd.DataFrame,
    policy: pd.DataFrame,
    transition: Dict[str, pd.DataFrame],
    penalty: pd.Series,
    rng: np.random.Generator,
) -> pd.DataFrame:
    """Generate synthetic rows matching real (sequence_id, step) layout."""
    meta = (
        real_df.sort_values([CLMN_SEQ_ID, CLMN_STEP])
            .groupby(CLMN_SEQ_ID, sort=False)
```

```

        .agg(length=(CLMN_STEP, "count"), phi0=(CLMN_PHI_SRC, "first"))
    )
    lengths = meta["length"].astype(int).to_numpy()
    phi0s = meta["phi0"].astype(str).to_numpy()
    seq_ids = meta.index.to_numpy()

    states = list(policy.index.astype(str))
    actions = list(policy.columns.astype(str))

    rows: List[dict] = []
    for sid, L, phi in zip(seq_ids, lengths, phi0s):
        phi = str(phi)
        if phi not in policy.index:
            phi = states[0]
        for step in range(1, int(L) + 1):
            prow = policy.loc[phi].to_numpy(dtype=float)
            a = _sample_from_row(rng, actions, prow)
            tmat = transition[a]
            labels = list(tmat.columns.astype(str))
            if phi in tmat.index:
                trow = tmat.loc[phi].to_numpy(dtype=float)
            else:
                trow = np.ones(len(labels), dtype=float) / len(labels)
            phi_next = _sample_from_row(rng, labels, trow)
            p_pen = float(penalty.reindex([a]).fillna(0.0).iloc[0])
            p_pen = min(max(p_pen, 0.0), 1.0)
            beta = int(rng.binomial(1, p_pen))
            rows.append(
                {
                    CLMN_SEQ_ID: int(sid),
                    CLMN_STEP: step,
                    CLMN_PHI_SRC: phi,
                    CLMN_PHI_DST: phi_next,
                    CLMN_ALPHA: a,
                    CLMN_BETA: beta,
                }
            )
            phi = phi_next

    return pd.DataFrame(rows)

```

A.6 main.py

`_drop_rows_randomly:`

```

def _drop_rows_randomly(
    df: pd.DataFrame,
    fraction_remove: float,
    rng: np.random.Generator,
) -> Tuple[pd.DataFrame, int]:
    """Return a copy of 'df' with 'round(fraction_remove * n)' rows removed."""
    n = len(df)
    if n <= 1 or fraction_remove <= 0.0:
        return df.copy(), 0
    n_remove = int(round(fraction_remove * n))
    n_remove = min(max(0, n_remove), n - 1)
    if n_remove <= 0:
        return df.copy(), 0
    drop_positions = rng.choice(n, size=n_remove, replace=False)
    out = df.drop(df.index[drop_positions]).reset_index(drop=True)
    return out, n_remove

```

`_run_tasks_1_to_4:`

```

def _run_tasks_1_to_4(
    df: pd.DataFrame,
    csv_path: Path,
    *,
    dataset_label: str,
    print_to_console: bool,
) -> PipelineRunResult:
    estimation = run_estimation(
        df,
        dataset_label=dataset_label,
        output_dir=None,
        print_to_console=print_to_console,
        print_penalty_per_state=False,
    )
    optimal = run_optimal_action(
        estimation=estimation,
        output_dir=None,
        print_to_console=print_to_console,
    )
    epsilon_res = run_epsilon_optimality(
        estimation=estimation,
        optimal=optimal,
        output_dir=None,
        print_to_console=print_to_console,
    )
    sim_res = run_simulation(
        estimation=estimation,
        csv_path=csv_path,
        output_dir=None,
        print_to_console=print_to_console,
    )
    return PipelineRunResult(
        label=dataset_label,
        n_rows=len(df),
        alpha_star=optimal.alpha_star,
        c_star=optimal.c_star,
        c_k_per_state=estimation.penalty_per_state.copy(),
        epsilon=epsilon_res.epsilon,
        min_p_alpha_star=epsilon_res.min_p_alpha_star,
        l1_action=sim_res.l1_action,
        l1_phi_src=sim_res.l1_phi_src,
        l1_phi_dst=sim_res.l1_phi_dst,
    )

```

`_print_c_k_per_state_runs:`

```

def _print_c_k_per_state_runs(runs: List[PipelineRunResult]) -> None:
    """Print per-state penalty tables for one or more pipeline runs."""
    lines = [
        _section("Task 6 --Model assumption check"),
        "c_k(phi_i) = P(beta=1 | alpha_k, phi_i)",
        "",
    ]
    for run in runs:
        lines.extend(_format_c_k_per_state_lines(run))
    print("\n".join(lines))

```

`_print_experiments_summary` (spread and table-delta helpers):

```

def _spread_across_states(c_k_per_state: pd.DataFrame) -> pd.Series:
    """Per action: max_phi c_k(phi) - min_phi c_k(phi) (P-model spread diagnostic)."""
    return c_k_per_state.max(axis=0) - c_k_per_state.min(axis=0)

```

```
def _max_abs_table_delta(reference: pd.DataFrame, other: pd.DataFrame) -> float:
    """Largest |c_k(phi, alpha) - reference| over aligned cells."""
    aligned = other.reindex_like(reference).fillna(0.0)
    return float(np.abs(aligned - reference).max().max())
```

```
)

lines.extend(
    [
        "Spread across states per action (max_phi c_k - min_phi c_k):",
        " Baseline: "
        + " ".join(f"{a}={baseline_spread[a]:.6f}" for a in actions),
    ]
)
if perturbed:
    if len(perturbed) > 1:
        spread_mean = perturbed_spreads.mean(axis=0)
        spread_std = perturbed_spreads.std(axis=0, ddof=1)
        lines.append(
            " Perturbed: "
            + " ".join(
                f"{a}=mean {spread_mean[i]:.6f} std {spread_std[i]:.6f}"
                for i, a in enumerate(actions)
            )
        )
        lines.extend(
            [
                "",
                "|c_k_per_state - baseline| max cell delta:",
                f" mean={table_deltas.mean():.6f} "
                f"std={table_deltas.std(ddof=1):.6f} "
                f"max={table_deltas.max():.6f}",
            ]
        )
    else:
        p_spread = _spread_across_states(perturbed[0].c_k_per_state)
        lines.append(
            " Perturbed: "
            + " ".join(f"{a}={p_spread[a]:.6f}" for a in actions)
        )
        lines.append(
            f"\n |c_k_per_state - baseline| max cell delta: "
            f"{table_deltas[0]:.6f}"
        )

print("\n".join(lines))
_print_c_k_per_state_runs([baseline, *perturbed])
```

run_all:

```
run_fingerprint(
    csv_path,
    output_dir=None,
    print_to_console=print_to_console,
)

# first run without perturbation
baseline = _run_tasks_1_to_4(
    df_full,
    csv_path,
    dataset_label=f"{csv_path.name} (baseline)",
    print_to_console=print_to_console,
```

```

)
results: List[PipelineRunResult] = [baseline]
if print_to_console:
    _print_run_results(baseline)

if perturbation_rate > 0:
    fraction = perturbation_rate / 100.0
    rng_master = np.random.default_rng(random_seed)
    for t in range(experiment_count):
        trial_seed = int(rng_master.integers(0, 2**31 - 1))
        rng = np.random.default_rng(trial_seed)
        df_p, n_removed = _drop_rows_randomly(df_full, fraction, rng)
        perturbation_run = _run_tasks_1_to_4(
            df_p,
            csv_path,
            dataset_label=f"{csv_path.name} (trial {t}, removed {n_removed})",
            print_to_console=print_to_console,
        )
        if print_to_console:
            _print_run_results(perturbation_run)
        results.append(perturbation_run)

    if print_to_console:
        _print_experiments_summary(
            csv_path=csv_path,
            n_full=len(df_full),
            perturbation_rate=perturbation_rate,
            experiment_count=experiment_count,
            baseline=baseline,
            perturbed=results[1:],
        )
    elif print_to_console:
        _print_c_k_per_state_runs([baseline])

```

A.7 util.py

Column constants, shared result types, and CSV loading:

```

# DataFrame / CSV column names (headerless file column order matches COLUMNS)
CLMN_SEQ_ID = "sequence_id"
CLMN_STEP = "step"
CLMN_PHI_SRC = "phi_src"
CLMN_PHI_DST = "phi_dst"
CLMN_ALPHA = "alpha"
CLMN_BETA = "beta"

COLUMNS = [
    CLMN_SEQ_ID,
    CLMN_STEP,
    CLMN_PHI_SRC,
    CLMN_PHI_DST,
    CLMN_ALPHA,
    CLMN_BETA,
]

@dataclass
class EstimationResult:
    """Structured return value of 'estimation.run_estimation'."""

    counts_phi_alpha: pd.DataFrame # N(phi_i, alpha_k)
    counts_phi_alpha_phi: Dict[str, pd.DataFrame] # N(phi_i, alpha_k, phi_j) per alpha

```

```

counts_alpha_beta: pd.DataFrame # N(alpha_k, beta)
policy: pd.DataFrame # P(alpha_k | phi_i)
transition: Dict[str, pd.DataFrame] # P(phi_j | phi_i, alpha_k) per alpha
penalty: pd.Series # c_k = P(beta=1 | alpha_k)
penalty_per_state: pd.DataFrame # c_k(phi_i) = P(beta=1 | alpha_k, phi_i)
empirical_penalty: pd.Series # groupby(CLMN_ALPHA)[CLMN_BETA].mean()
checks: Dict[str, bool] # named pass/fail flags
states: list
actions: list
n_rows: int

```

```

@dataclass
class OptimalActionResult:
    """Structured return value of ‘optimal_action_identification.run_optimal_action‘. """

```

```

c_star: float # its penalty probability (the minimum c_k)
ties: List[str] # all actions tied at the minimum (usually just [alpha_star])
ranking: pd.DataFrame # actions sorted by c_k (ascending), columns: c_k, rank
penalty: pd.Series # the c_k vector echoed back (action -> c_k)
source: str # "estimation-result" or absolute path to penalty.csv

```

```

def load_vsa_dataset(csv_path: Union[Path, str]) -> pd.DataFrame:
    """Load the headerless UTF-8 VSA interaction CSV.

    Each row is
    ‘(CLMN_SEQ_ID, CLMN_STEP, CLMN_PHI_SRC, CLMN_PHI_DST, CLMN_ALPHA, CLMN_BETA)’
    with integer ‘CLMN_SEQ_ID’, ‘CLMN_STEP’, and ‘CLMN_BETA’; state and
    action columns stay as strings (e.g. Greek labels).
    """
    csv_path = Path(csv_path)
    df = pd.read_csv(csv_path, header=None, names=COLUMNS, encoding="utf-8")
    df[CLMN_SEQ_ID] = df[CLMN_SEQ_ID].astype(int)
    df[CLMN_STEP] = df[CLMN_STEP].astype(int)
    df[CLMN_BETA] = df[CLMN_BETA].astype(int)
    return df

```